



Deep learning

I. Preprocesamiento de los datos

Ejercicio 1

Con el repositorio brooklyn_sales_map que esta disponible en el siguiente enlace:

https://www.kaggle.com/datasets/tianhwu/brooklynhomes2003to2017?select=brooklyn_sales_map.csv

Realice lo siguiente en un Notebook de Jupyter:

1. Cargue los datos en Python usando Pandas y muestre los primeros 10 registros
2. Indique cuales son las dimensiones del dataframe (renglones, columnas)
df.shape
3. Identifique que porcentaje de valores faltantes tiene cada columna.
4. Elimine todas las columnas que no tienen al menos el 75% de valores no nulos.
5. Elimine todos los registros que tienen NaN en la columna 'building_class_category'
6. Ejecute el siguiente comando para consultar estadísticas del dataframe.
df.describe()
7. El dataframe contiene algunas columnas duplicadas pues se generó al combinar dos fuentes de datos y algunas de ellas son irrelevantes para el análisis que realizaremos. Elimine estas columnas con el siguiente comando:

```
df.drop(columns=['Unnamed: 0', 'borough', 'Borough', 'AreaSource', 'block', 'LotArea', 'XCoord', 'YCoord', 'BldgArea', 'BldgClass', 'Easements', 'sale_date', 'ZipCode', 'CT2010', 'CB2010', 'Tract2010', 'Sanborn', 'HealthCent', 'ZoneMap', 'SchoolDist', 'TaxMap', 'YearAlter1', 'YearAlter2', 'CondoNo', 'APPBBL', 'PLUTOMapID', 'MAPPLUTO_F', 'Version', 'land_sqft', 'Address', 'UnitsRes', 'UnitsTotal', 'ComArea', 'ResArea', 'OfficeArea', 'RetailArea', 'GarageArea', 'StrgeArea', 'FactryArea', 'OtherArea'], axis=1, inplace=True)
```

df.shape

8. Las columnas de alta cardinalidad, con un gran número de categorías, pueden aumentar la complejidad de nuestro modelo. Para mitigar esto, eliminaremos columnas de alta cardinalidad, con la excepción de Neighborhood (barrio), Building Class(clase de edificio) y tax_class (tipo de impuestos). Estas tres columnas tienen un impacto potencial en nuestra variable objetivo y se conservarán.



```
cat_cols = df.select_dtypes('object').columns
```

```
k = 0
```

```
cardinality_cols = []
```

```
for col in cat_cols:
```

```
    value_counts = df[col].value_counts(normalize=True).round(2)
```

```
    if len(value_counts) > 5 or len(value_counts) < 2:
```

```
        cardinality_cols.append(col)
```

```
    else:
```

```
        print("=====")
```

```
        print(col)
```

```
        print(value_counts)
```

```
cardinality_cols.remove('neighborhood')
```

```
cardinality_cols.remove('building_class_at_sale')
```

```
cardinality_cols.remove('tax_class')
```

```
print(f"Removed Columns: {cardinality_cols}")
```

```
df.drop(columns=cardinality_cols, axis=1, inplace=True)
```

9. Revise que porcentaje de datos tiene cada categoría de neighborhood (barrio).

```
df['neighborhood'].value_counts(normalize=True).round(2)
```

10. Como podrá observar la columna "barrio" tiene una cantidad excesiva de categorías, con 61 en total. Para simplificar los datos, agruparemos las categorías con menos del 1% de representación en una categoría "OTRONEIG".

```
import numpy as np
```

```
neighborhood = df['neighborhood'].value_counts(normalize=True).round(2)
```

```
idx = np.where(neighborhood > 0.01)[0].max()
```

```
places = neighborhood.index[:idx]
```

```
df['neighborhood'] =
```

```
df['neighborhood'].apply(lambda x: x if x in places else 'OTRONEIG')
```

```
df['neighborhood'].value_counts(normalize=True)
```

Aquí hacemos uso de una función lambda, estas son funciones anónimas que simplifican la sintaxis de una función, pues solo se indica lambda PARAMETROS: CODIGO

Puede encontrar una explicación más detallada [AQUI](#)

Apply aplica la función indicada a cada uno de los valores de la columna. (Exactamente como lo haría con un ciclo).

11. Repita los pasos 9 y 10 con la columna "building_class_at_sale" agrupando a las categorías que tienen menos del 1% en 'OTRACLASS'.



12. Elimine las columnas de varianza baja con el siguiente código:

```
num_cols = df.select_dtypes('number').columns
```

```
low_var_cols = []
for col in num_cols:
    scaled = (df[col] - df[col].mean()) / df[col].std()
    variance = scaled.var().round(4)
    if variance == 0 or df[col].std() == 0:
        low_var_cols.append(col)
# else:
#     print(col, variance)
```

```
df.drop(columns=low_var_cols, axis=1, inplace=True)
print(f'Removed Columns: {low_var_cols}')
print(df.shape, len(low_var_cols))
```

13. Realice un histograma y un diagrama de cajas del precio de venta (sale_price)

14. No tiene sentido que algunas propiedades se hayan vendido en \$0 dólares, de hecho por la ubicación descartaremos aquellas que tengan un precio inferior a 100 mil dólares.

```
mask = (df['sale_price'] > 100000)
df = df[mask]
```

15. Obtenga los valores de los cuartiles 1 y 3 y del rango intercuartil IQR del sale_price.

16. Obtenga los valores de los bigotes inferior y superior.

17. Elimine los outliers del dataframe:

```
df.drop(df.loc[(df['sale_price'] < BI) | (df['sale_price'] > BS)].index, inplace=True)
```

Explicación:

```
df.loc[(df['sale_price'] < BI) | (df['sale_price'] > BS)]
```

extrae del dataframe aquellos registros que cumplen la condición entre corchetes (En este caso los outliers)

.index

Obtiene los índices de los registros extraídos

df.drop

Genera una copia del dataframe eliminando los registros con los índices indicados

Inplace=True

En lugar de realizar una copia hace la eliminación en el conjunto original.



18. Vuelva a realizar el histograma y el diagrama de cajas para ver la actual distribución del precio.
19. Obtenga la edad de la propiedad usando las columnas `year_built` y `YearBuilt`

```
df['age'] = 2023 - (df['year_built'] + df['YearBuilt']) // 2
```

```
df.drop(['zip_code', 'year_built', 'YearBuilt', 'year_of_sale'], axis=1, inplace=True)
```
20. Aunque `ProxCode` es numérico conviene remplazar los valores faltantes por la **moda**. Haga lo mismo con `IrrLotCode` y `building_class_at_sale`.
21. Verificaremos cuáles son las categorías y cuántos valores nulos poseen las columnas `tax_class` y `tax_class_at_sale`

```
print(df['tax_class'].unique())
```

```
print(df['tax_class'].isnull().sum())
```

```
print(df['tax_class_at_sale'].unique())
```

```
print(df['tax_class_at_sale'].isnull().sum())
```
22. Remplazamos los valores faltantes de `tax_class` con los de `tax_class_at_sale` y eliminamos a esta última.

```
df['tax_class'].fillna(df['tax_class_at_sale'], inplace=True)
```

```
del df['tax_class_at_sale']
```
23. La columna `tax_class` ahora tiene valores de texto y enteros, lo cual es un problema para su manejo, convierta todos los valores a texto con el siguiente código:

```
df['tax_class'] = df['tax_class'].map({'1A':'1A', '1B':'1B', '1C':'1C', '2A':'2A', '2B':'2B', '2C':'2C', '4':'4', '3':'3', '2':'2', '1':'1', '3':'3', '4':'4', '2':'2', '1':'1'})
```

```
print(df['tax_class'].unique())
```
24. Verifique el tipo de las columnas

```
df.dtypes
```
25. `SplitZone` es booleana (Y/N) pero como tiene valores faltantes NaN la identifica como categórica.

```
print(df['SplitZone'].unique())
```

```
df['SplitZone'] = df['SplitZone'].fillna(value='N')
```

```
df['SplitZone'] = df['SplitZone'].astype('category')
```

```
df['SplitZone'] = df['SplitZone'].cat.codes
```

```
print(df['SplitZone'].unique())
```
26. Reemplace los valores faltantes de las columnas numéricas por la **mediana** (**median**).
27. Compruebe que ya no queda ningún valor NaN



28. Convierta la columna lrrLotCode a numérica con el siguiente código:

```
df['lrrLotCode']= df['lrrLotCode'].astype('category')
cat_columns = df.select_dtypes(['category']).columns
print(cat_columns)
df[cat_columns] = df[cat_columns].apply(lambda x: x.cat.codes)
```

29. Genere un dataframe a partir del dataframe actual ignorando las columnas categóricas y la variable objetivo(sale_price).

El análisis de correlación y pruebas de VIF le indicarán que columnas debe eliminar, hagalo en ambos dataframes (df, X)

```
X = df.drop([ 'sale_price', 'neighborhood', 'tax_class', 'building_class_at_sale',
'ProxCode' ], axis=1)
```

30. Obtenga la matriz de correlación del dataframe X y muéstrela en su formato tidy.

31. Genere el mapa de calor de la matriz de correlación con un figsize=(22, 22).

32. Elimine las columnas que están altamente correlacionadas con el siguiente código:

```
corr = X.corr()
upper = corr.where(np.triu(np.ones(corr.shape), k=1).astype(np.bool_))
drop = [column for column in upper.columns if any(np.abs(upper[column]) > 0.7)]
print(f"columns dropped are: {drop}")
df.drop(columns=drop, axis=1, inplace=True)
X.drop(columns=drop, axis=1, inplace=True)
```

33. Calcule el factor de inflación de la varianza (VIF) de las columnas de X.

```
from statsmodels.stats.outliers_influence import variance_inflation_factor
vif = pd.DataFrame()
vif["VIF Factor"] = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
vif["features"] = X.columns
vif.round(1)
```

34. Observe cual es la característica (predictor) con el VIF más alto (SanitBoro con 4661.6) y elimínela de los dataframes X y df.

```
del df ['SanitBoro']
del X ['SanitBoro']
```

35. Repita los pasos 33 y 34 hasta que todas las características tengan un VIF menor a 5. Elimine solo una columna en cada iteración.

***NOTA:** Esta prueba solo puede aplicarse a columnas numéricas, por eso usamos X.

En total se eliminarán **6** columnas en este ejercicio incluyendo a SanitBoro



36. Grafique la distribución de los datos con el siguiente código

```
viz_df = df.copy()
median_home_price = viz_df['sale_price'].median().round(2)
print(median_home_price)
k = 0
plt.figure(figsize=(20, 30))
for col in viz_df.drop('sale_price', axis=1).columns:
    plt.subplot(6, 3, k + 1)
    if viz_df[col].dtype == 'float64' or viz_df[col].dtype == 'int8':
        sns.scatterplot(data=viz_df, x=col, y='sale_price')
        plt.title(f"Sale Price Vs {col}")
    else:
        group = viz_df[[col, 'sale_price']].groupby(by=col).median()
        sns.barplot(data=group, x=group.index, y='sale_price')
        plt.axhline(y=median_home_price, label='Median Home Price', color='black',
linestyle='--', linewidth=0.7)
        plt.title(f"Avg Home Price by {col}")
        plt.legend()
    k += 1
plt.tight_layout()
```

37. Convierta la columna neighborhood al formato OneHot con el siguiente código:

```
data = pd.get_dummies(df.neighborhood)
df = pd.concat([df.drop("neighborhood",axis="columns"), data], axis="columns")
df.head()
```

pd.get_dummies genera un dataframe con la columna en su formato OneHot (variables dummy)

pd.concat concatena 2 dataframes, axis="columns" indica que la unión es horizontal

38. Convierta las columnas tax_class, ProxCode y building_class_at_sale al formato OneHot.

39. Muestre las dimensiones del dataframe df. Deberían ser 73 características incluyendo sale_price.

40. Normalice las columnas con el siguiente código:

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df_scaled = scaler.fit_transform(df.to_numpy())
df_scaled = pd.DataFrame(df_scaled,columns=df.columns)
```

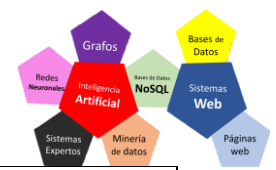


41. El formato One-hot generó 3 columnas llamadas 1.0, 2.0 y 3.0 (puede verificarlo imprimiendo `df_scaled.columns`) que como podrá notar no son cadenas de texto para evitar problemas con los modelos de machine learning ejecute el siguiente comando:

```
df_scaled.columns = df_scaled.columns.astype(str)
```

42. Analicemos las características con las que trabajaremos:

Característica	Descripción
sale_price	precio de venta de la propiedad (Objetivo)
neighborhood	barrio de la propiedad.
tax_class	clase fiscal de la propiedad (impuestos)
lote	Área total del lote fiscal, expresada en pies cuadrados redondeados al entero más cercano.
residential_units	Unidades Residenciales (casas, departamentos) La suma de unidades residenciales en todos los edificios en el lote fiscal.
commercial_units	Unidades comerciales (oficinas, ventas, etc.)
SplitZone	Indicador de límite dividido Un código que indica si el lote fiscal se divide entre varias características de zonificación. El indicador de límite dividido es igual a "Y" si el lote fiscal tiene un valor para distrito de zonificación 2, Superposición comercial 2 o Límite de distrito especial 2.
NumBldgs	Número de edificios en el lote fiscal.
NumFloors	Número de pisos de la propiedad
BldgFront BldgDepth	Medidas de la propiedad frente y profundidad.
ProxCode	Código de proximidad Un código que describe la relación física del edificio con los edificios vecinos.



	1. No disponible 2. Independiente 3. Semiadjunto 4. Adjunto
IrrLotCode	Código que indica si el lote fiscal tiene una forma irregular(Y) o no (N).
Age	Edad de la propiedad, calculada a partir del año de construcción (yearbuild)